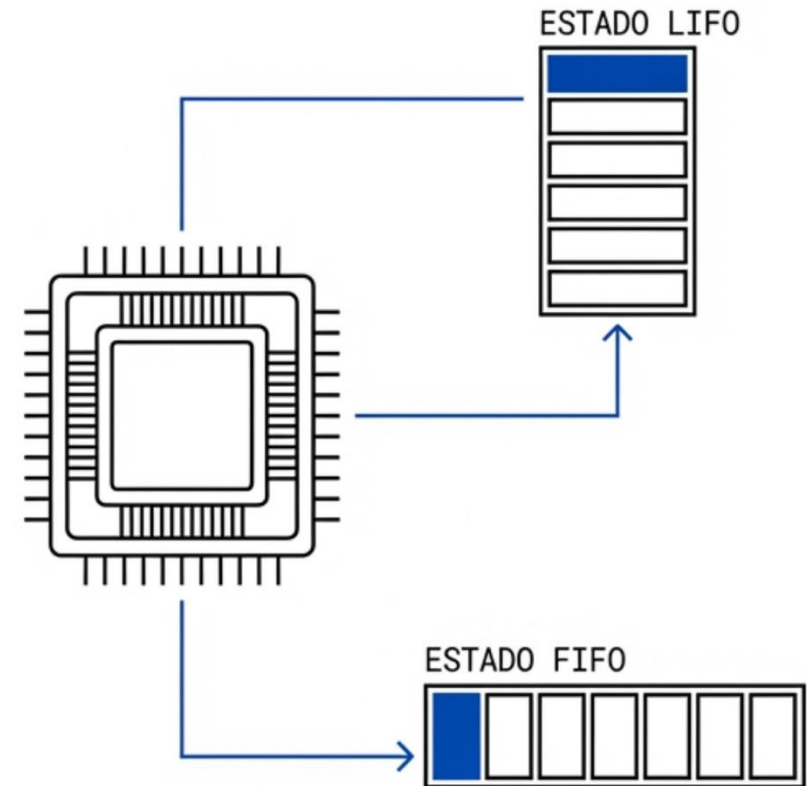


Pilas y Colas





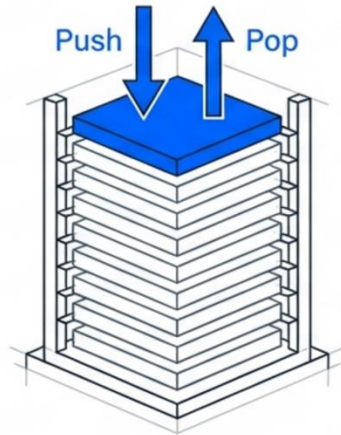
Objetivos

- Distinguir las estructuras de pilas y colas, identificando los escenarios algorítmicos donde su aplicación es computacionalmente óptima.
- Implementar pilas y colas empleando arreglos y listas enlazadas.
- Evaluar los compromisos arquitectónicos, principalmente la localidad de cache frente a la flexibilidad de redimensionamiento.
- Resolver el problema de la degradación espacial, inherente a las colas estáticas lineales.
- Demostrar que las operaciones de inserción y recuperación ocurren en $\mathcal{O}(1)$.

Motivación

Las pilas y colas son los pilares fundamentales sobre los que se construyen los sistemas operativos actuales.

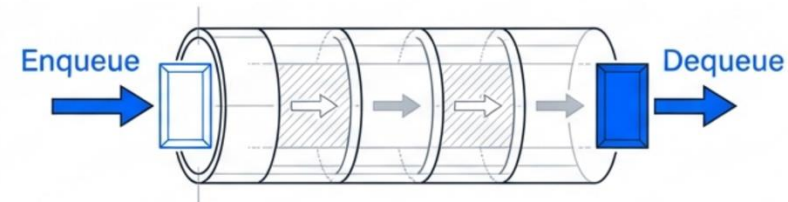
LIFO (Last-In-First-Out)



Rol del Sistema: Memoria y Estado.
Controla la profundidad de la ejecución.

Aplicaciones: Evaluación de expresiones,
Pila de llamadas (Call Stack).

FIFO (First-In-First-Out)

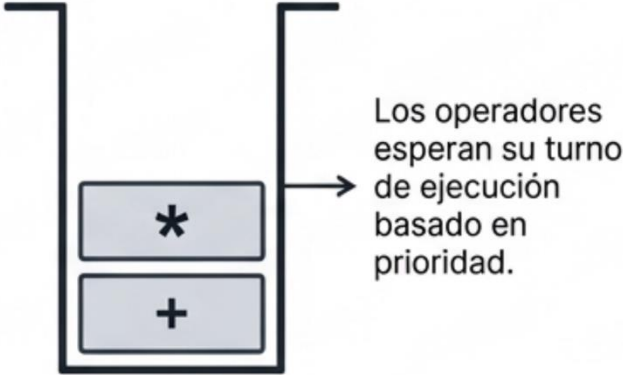


Rol del Sistema: Tiempo y Sincronización.
Administra la amplitud de los procesos.

Aplicaciones: Planificadores del SO (OS
Schedulers), Buffers de Entrada/Salida.

Evaluación de expresiones

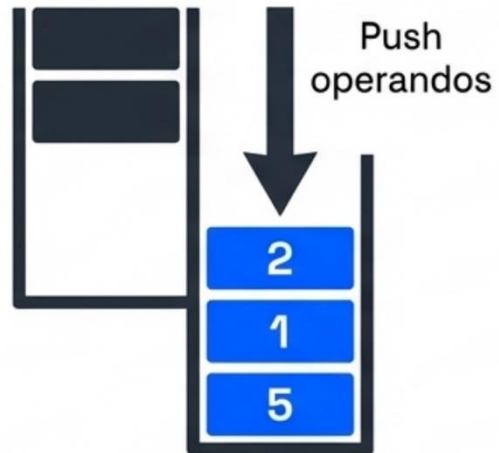
Para evaluar una expresión matemática compleja, el ordenador requiere resolver problemas de precedencia y asociatividad. La **Notación Postfija** elimina la ambigüedad y la necesidad de paréntesis.

Matemática Humana (Infija)	Transformación (Acción de la Pila)	Matemática de Máquina (Postfija)
$3 + 4 * 2$ Ambigüedad: Requiere carga cognitiva para evaluar precedencia.	 Los operadores esperan su turno de ejecución basado en prioridad.	$3 \ 4 \ 2 \ * \ +$ Ejecución lineal $O(1)$ sin retrocesos.

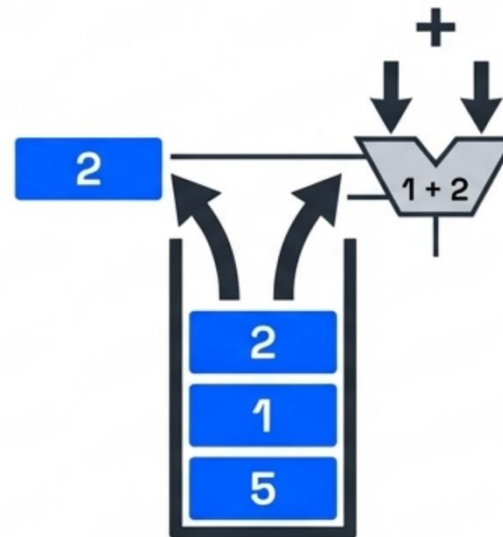
Evaluación de expresiones

5 1 2 + 4 * + 3 -

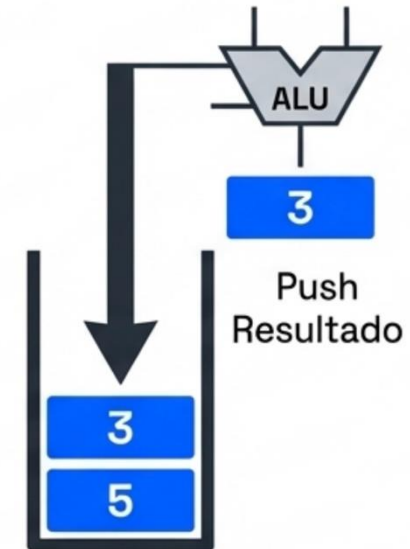
Paso 1 (Push):



Paso 2 (Pop & Execute):



Paso 3 (Push Result):



La naturaleza LIFO de la pila garantiza que los operandos correctos estén siempre disponibles exactamente cuando el operador los necesita, en tiempo $O(1)$.

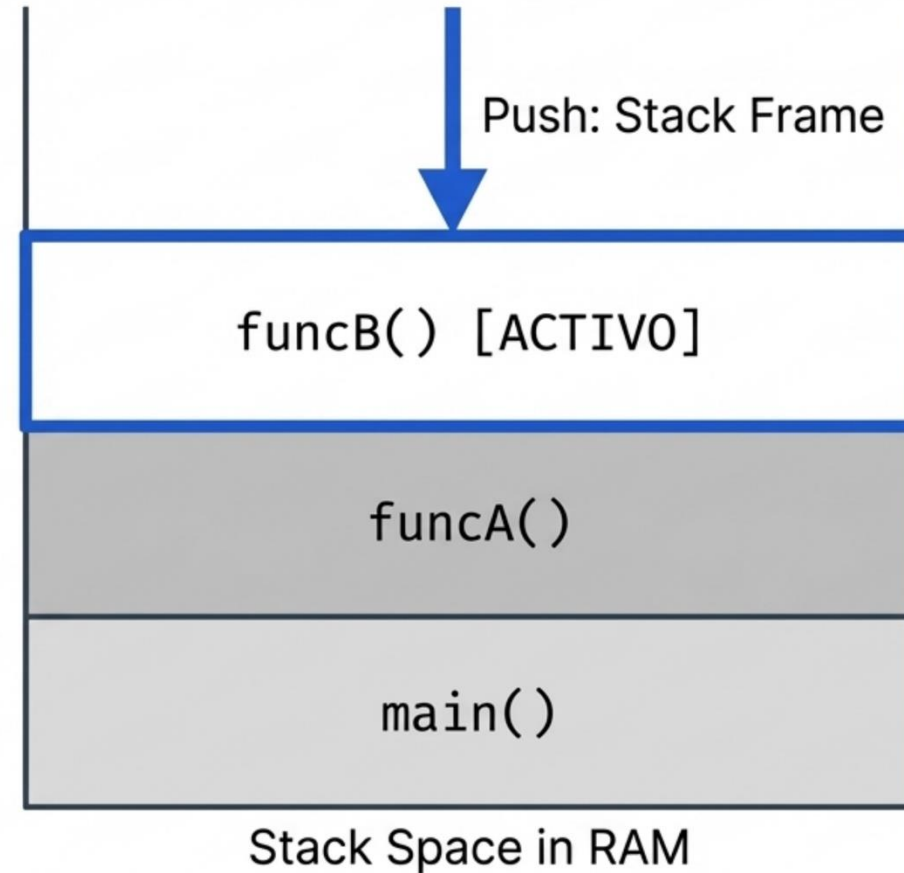
Call Stack

Cada vez que un programa en C++ invoca una función, se empuja un “**Stack Frame**” a la pila de ejecución del sistema.

Es el engranaje mecánico que hace posible la recursión geométrica sin que el hardware colapse.

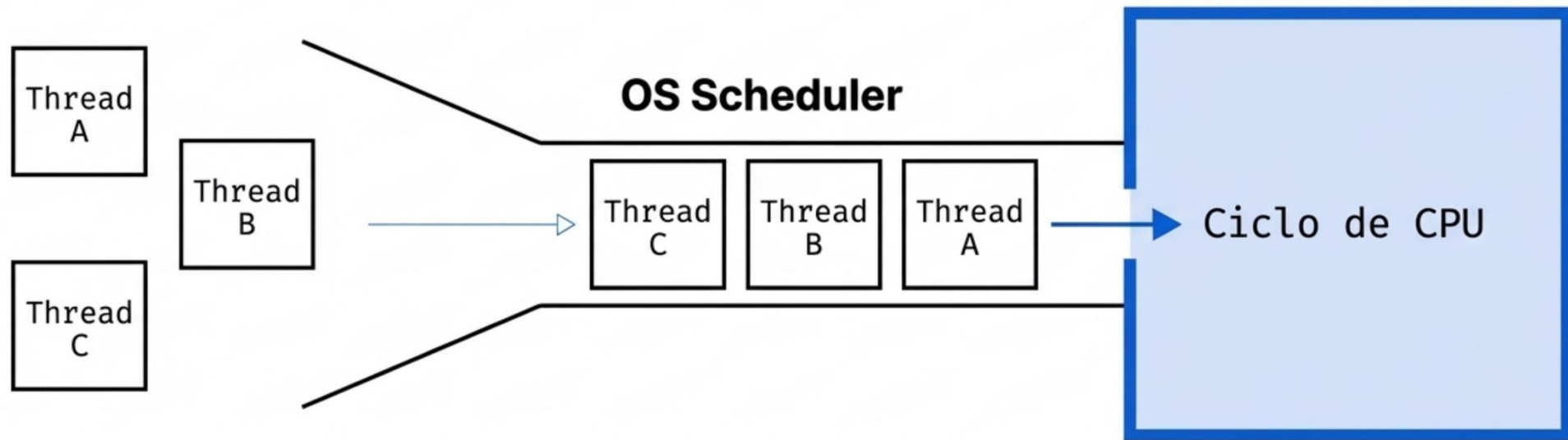
Almacena Variables Locales

Guarda Dirección de Retorno



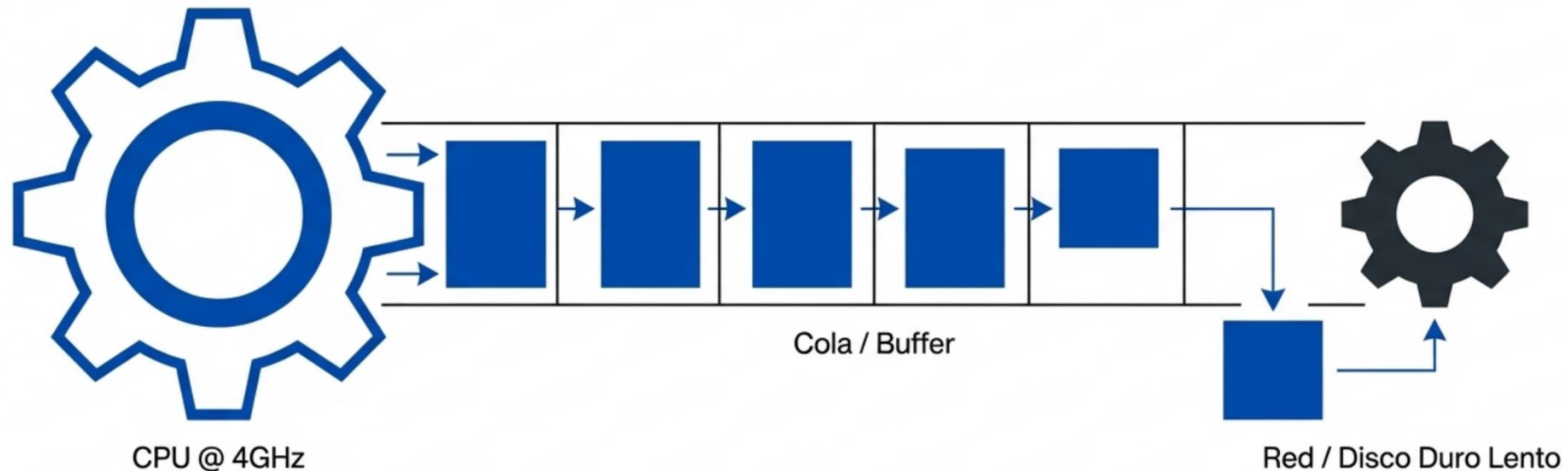
Scheduler

1. **El Contexto:** Los recursos de CPU son finitos frente a un infinito teórico de procesos.
2. **Mecánica FIFO:** Las tareas que solicitan tiempo de CPU se forman en colas rígidas.
3. **Garantía de Hardware:** Asegura que ningún proceso sufra inanición matemática (starvation).



Buffers de Entrada/Salida (I/O)

Transferencia Asíncrona (Spooling): Concilia la discrepancia física cuando una CPU ultra-rápida transmite a periféricos de alta latencia. Las colas actúan como amortiguadores temporales (buffers) para evitar la pérdida de paquetes (packet drop).

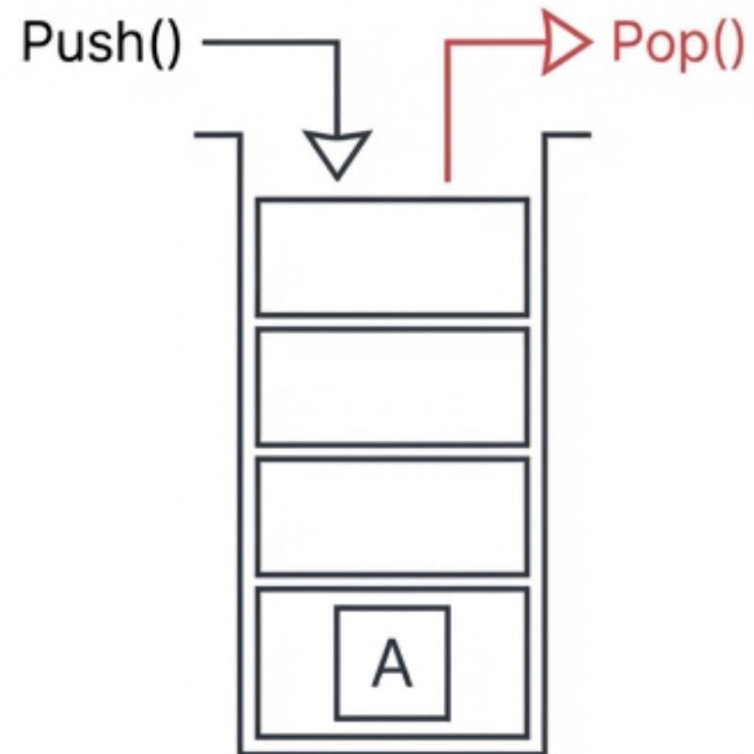


Pilas vs colas

	Pila (LIFO)	Cola (FIFO)
Dominio del SO	Micro-Estado	Macro-Estado
Flujo de Control	Ejecución Anidada / Recursión	Planificación Secuencial / Paralelismo
Gestión de Memoria	Liberación Automática (Scope)	Amortiguación Asíncrona (Spooling)
Ventaja Algorítmica	Rastreabilidad y Retroceso	Equidad y Prevención de Inanición

Síntesis Sistémica: Las Pilas deciden QUÉ se ejecuta ahora y a dónde se regresa. Las Colas deciden QUIÉN sigue en el ecosistema global.

Pila (Stack) - LIFO



Validación de sintaxis

Algoritmo ValidarSintaxis(cadena_codigo):

1. Inicializar una Pila vacía llamada 'pila_delimitadores'

2. Para cada caracter 'c' en 'cadena_codigo' hacer:

a. Condición de Apertura:

Si 'c' es '(', '[' o '{':

PUSH(pila_delimitadores, c)

b. Condición de Cierre:

Si 'c' es ')', ']' o '}':

// Caso de falla 1: Subdesbordamiento lógico (cierre sin apertura)

Si pila_delimitadores está vacía:

Retornar FALSO

// Recuperar el último delimitador abierto

caracter_cima = POP(pila_delimitadores)

// Caso de falla 2: Discordancia estructural

Si caracter_cima NO hace pareja correcta con 'c':

Retornar FALSO

// (ej. cima es '(' pero 'c' es ']')

3. Fin del bucle

// Caso de falla 3: Bloques de código sin cerrar

Si pila_delimitadores NO está vacía:

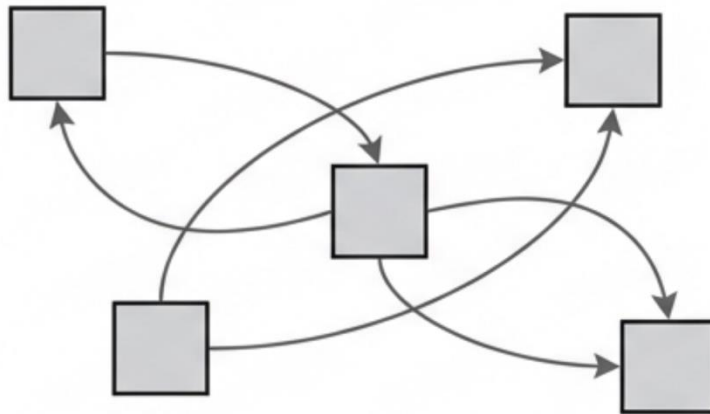
Retornar FALSO

// Si la cadena se procesó completa y la pila quedó vacía, la sintaxis es perfecta

Retornar VERDADERO

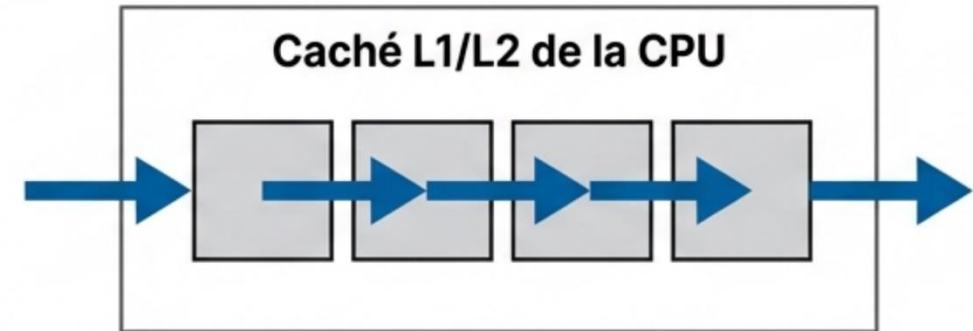
Pila estática

Nodos Dispersos (Listas Enlazadas)



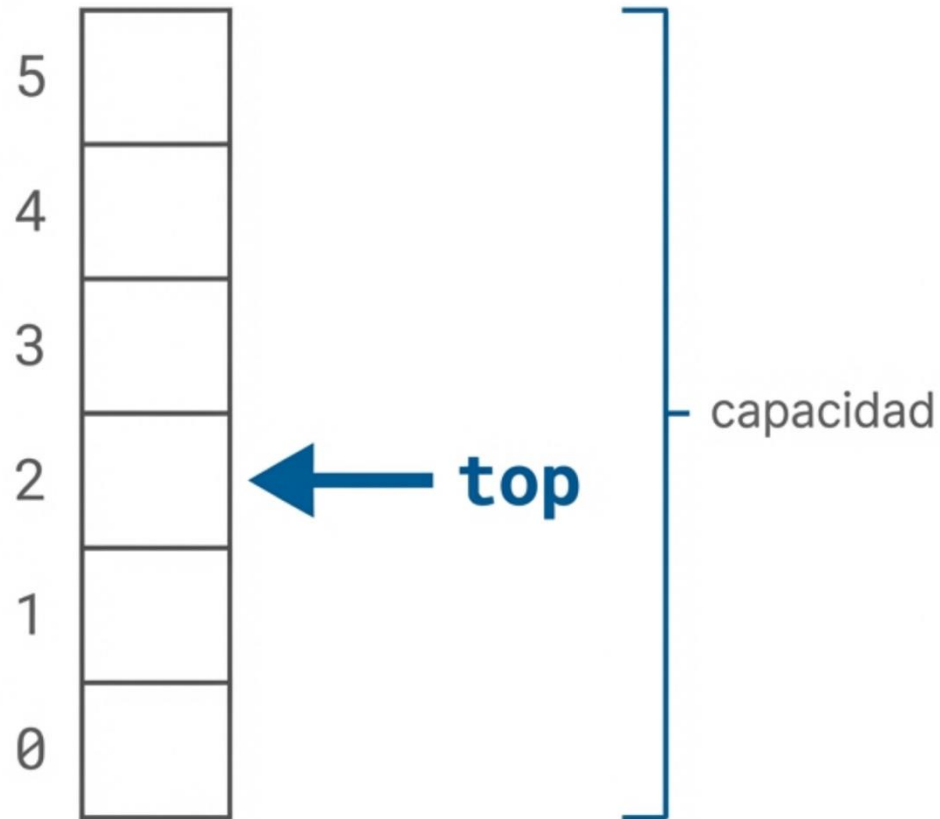
Fragmentación de memoria. Saltos de puntero costosos en ciclos de reloj que rompen la secuencia de lectura de la CPU.

Bloque Contiguo (Arreglo Estático)



Los elementos se almacenan en bloques de memoria adyacentes.

Pila estática



La arquitectura estática depende estrictamente de tres variables de estado:

- **arreglo:** La infraestructura física. Un bloque unidimensional de almacenamiento.
- **capacidad:** El límite estructural estricto predefinido (N).
- **top:** El apuntador lógico. Un índice entero que dicta el estado de vida de la pila. Se inicializa en -1, representando la ausencia total de datos (pila vacía).

Pila estática (push)

Algoritmo PUSH(Pila, elemento):

Si Pila.top == Pila.capacidad - 1 **entonces**

Lanzar Excepción 'Stack Overflow' (Desbordamiento)

Pila.top = Pila.top + 1

Pila.arreglo[Pila.top] = elemento

→ El índice lógico top SIEMPRE se incrementa antes de la asignación de memoria.

Pila estática (pop)

```
Algoritmo POP(Pila):
```

```
    Si Pila.top == -1 entonces
```

```
        Lanzar Excepción 'Stack Underflow' (Subdesbordamiento)
```

```
    valor = Pila.arreglo[Pila.top]
```

```
    Pila.top = Pila.top - 1
```

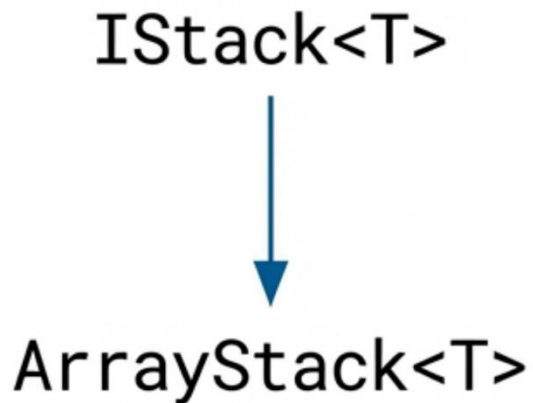
```
    Retornar valor
```

→ El valor físico no se borra de la memoria; el post-decremento de top simplemente lo invalida lógicamente.

Pila estática: Límites y excepciones

Operación	Condición Lógica	Significado Físico	Excepción en C++
PUSH	<code>top == capacidad - 1</code>	Estructura sin memoria disponible	<code>std::overflow_error</code>
POP	<code>top == -1</code>	Estructura sin elementos (Vacía)	<code>std::underflow_error</code>
TOP	<code>top == -1</code>	Lectura en el vacío	<code>std::underflow_error</code>

Pilas (abstracción en c++)

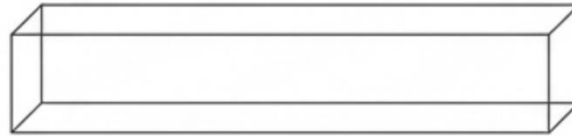


```
#include <iostream>
#include <stdexcept>

template <typename T>
class ArrayStack : public IStack<T> {
private:
    T* array;           // Puntero al bloque contiguo
    size_t capacity;    // Límite máximo
    int topIndex;       // Apuntador lógico
    ...
};
```

Pila estática: Gestión de memoria

Memoria Dinámica: new T[]



ArrayStack()

~ArrayStack()

```
explicit ArrayStack(size_t cap = 100)
: capacity(cap), topIndex(-1) {
    array = new T[capacity];
}
```

```
~ArrayStack() override {
    delete[] array;
}
```

La memoria dinámica se vincula indisolublemente al ciclo de vida del objeto. Al instanciar, se asegura el bloque (new). Al destruir, se garantiza la limpieza (delete), previniendo fugas de memoria (memory leaks).

Pila estática (push / pop)

```
void push(const T& element) override {  
    if (topIndex == static_cast<int>(capacity) - 1)  
        throw std::overflow_error("Stack Overflow");  
  
    array[++topIndex] = element; // Pre-incremento O(1)  
}
```

```
T pop() override {  
    if (isEmpty())  
        throw std::underflow_error("Stack Underflow");  
  
    return array[topIndex--]; // Post-decremento O(1)  
}
```

Pila estática (observadores)

Acceso a la Cima

```
T top() const override {  
    ... return array[topIndex];  
}
```

Validación de Vacío

```
bool isEmpty() const override {  
    return topIndex == -1;  
}
```

Cálculo de Tamaño

```
size_t getSize() const override {  
    return topIndex + 1;  
}
```

Pila dinámica

Una vez identificado el cuello de botella espacial, vamos a introducir la arquitectura dinámica. Su única restricción es la memoria física del sistema.

El Cuello de Botella

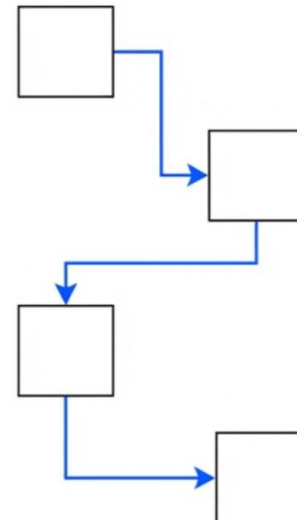
Stack / Array



Límite de capacidad rígido. Requiere redimensionamiento costoso.

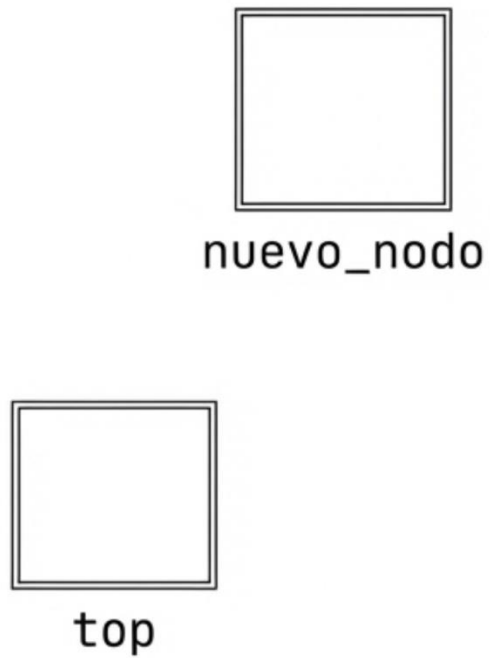
La Solución Dinámica

Heap (Montón)

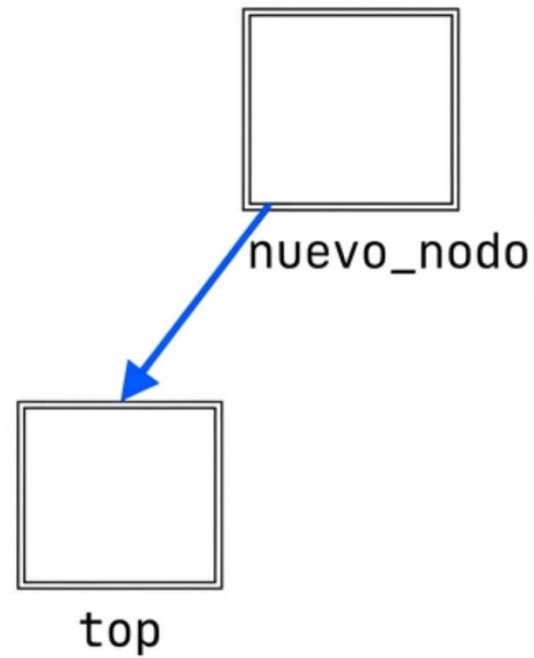


Memoria asignada individualmente, conectada mediante punteros. Sin restricciones artificiales.

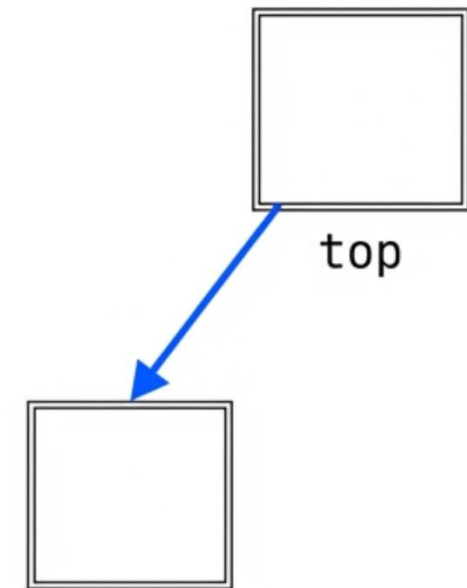
Pila dinámica (push)



1. Asignar memoria

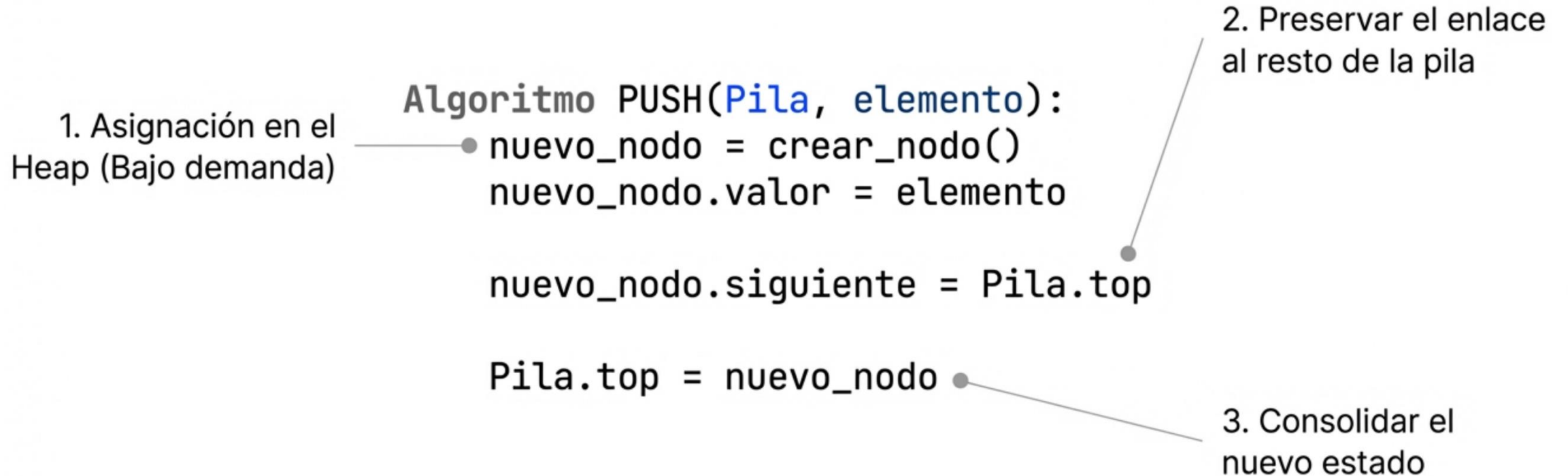


2. Enlazar a la cima actual



3. Actualizar puntero principal

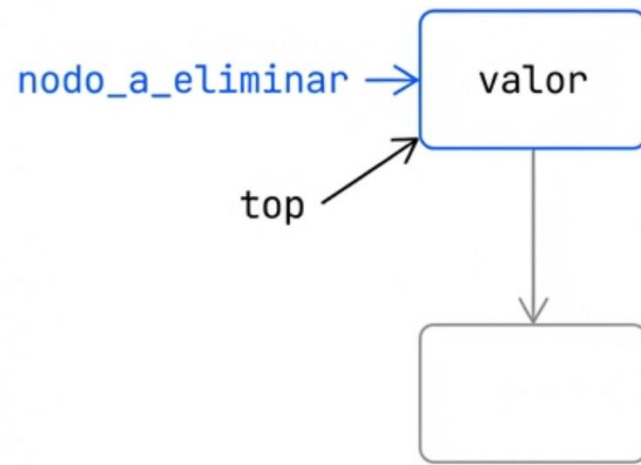
Pila dinámica (push)



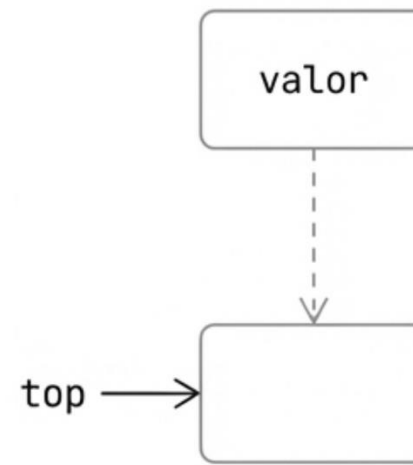
Pila dinámica (pop)



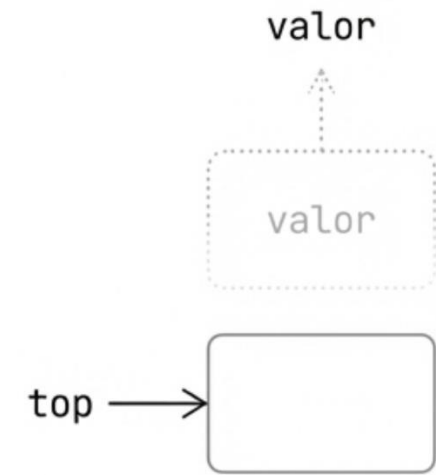
Riesgo: Stack Underflow
(pila vacía)



1. Identificar cima



2. Actualizar cima



3. Liberar y retornar

Pila dinámica (pop)

● Validación de
estado imperativa

Algoritmo POP(*Pila*):

Si *Pila.top* es NULO entonces

● Lanzar Excepción "Stack Underflow"

nodo_a_eliminar = *Pila.top*

valor_extraido = *nodo_a_eliminar.valor*

Pila.top = *Pila.top.siguiente* ●

Reasignación de
la cima

liberar_memoria(*nodo_a_eliminar*) ●

Retornar *valor_extraido*

● Prevención estricta
de Memory Leaks

Pila dinámica

```
template <typename T>
class LinkedStack : public IStack<T> {
private:
    struct Node {
        T data;
        Node* next;
        Node(const T& val, Node* nextNode = nullptr)
            : data(val), next(nextNode) {}
    };
    Node* topNode;
    size_t currentSize;
```

Encapsulamiento de Arquitectura:

La estructura Node se anida dentro de la clase privada para evitar la contaminación del espacio de nombres global. Un constructor directo agiliza la inicialización.

Pila dinámica: Gestión de memoria

```
public:
    LinkedStack() : topNode(nullptr),
        currentSize(0) {}

    ~LinkedStack() override {
        while (!isEmpty()) {
            pop();
        }
    }
}
```

Destrucción Segura:

El Destructor recorre la lista y libera cada nodo distribuido. Al reutilizar pop(), se centraliza la lógica de borrado (delete), garantizando cero fugas de memoria al finalizar el ciclo de vida del objeto.

Pila dinámica (push / pop)

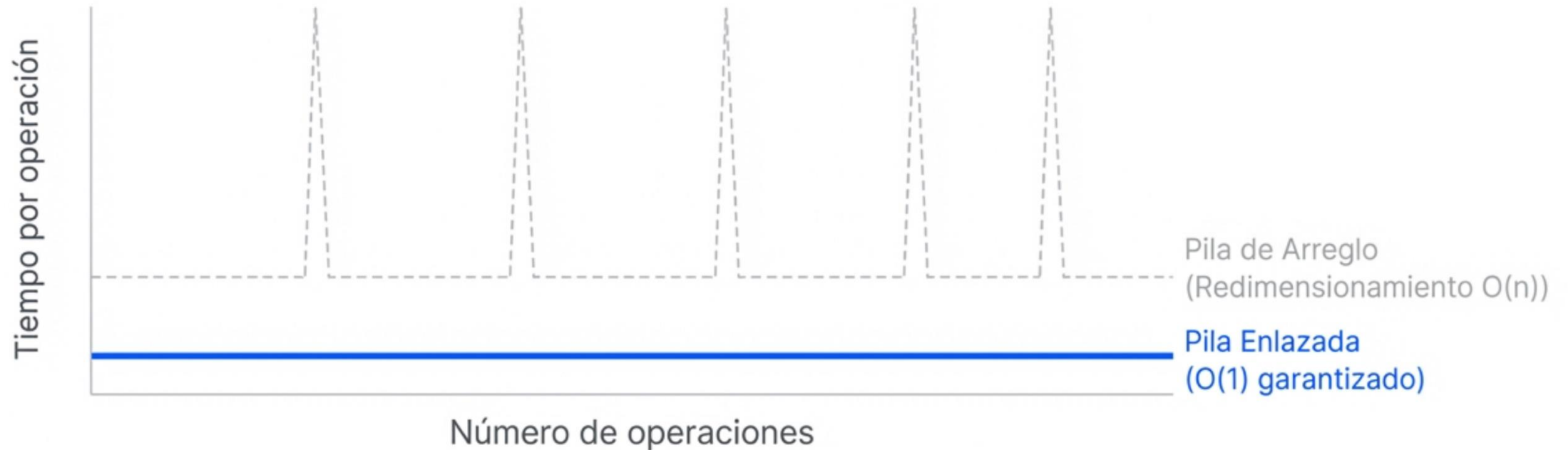
```
void push(const T& element) override {  
    topNode = new Node(element, topNode);  
    currentSize++;  
}
```

Asignación dinámica en
el heap (lanza
std::bad_alloc si falla)

```
T pop() override {  
    // ... underflow check ...  
    Node* nodeToDelete = topNode;  
    T poppedValue = nodeToDelete->data;  
    topNode = topNode->next;  
    delete nodeToDelete;  
    currentSize--;  
    return poppedValue;  
}
```

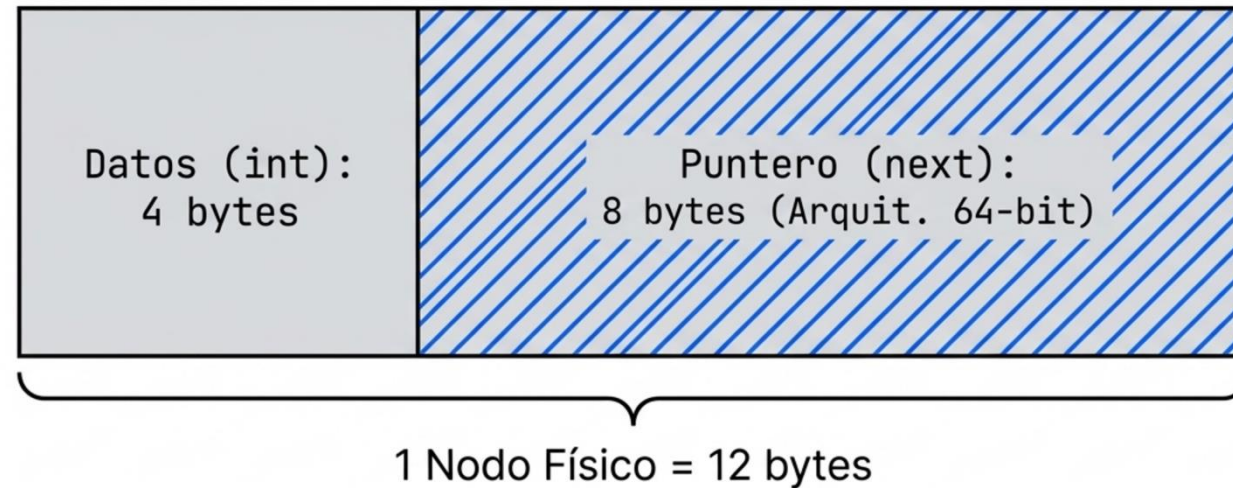
Devolución de la
memoria al Sistema
Operativo

ArrayStack vs LinkedStack



A diferencia de la redimensión de arreglos dinámicos, push y pop mantienen un tiempo garantizado de $O(1)$ en el peor de los casos. Cero copias masivas de datos.

ArrayStack vs LinkedStack



La penalización invisible. Por cada elemento insertado, la infraestructura de enlace puede consumir el doble de memoria física que la carga útil (payload) en sí misma.

ArrayStack vs LinkedStack

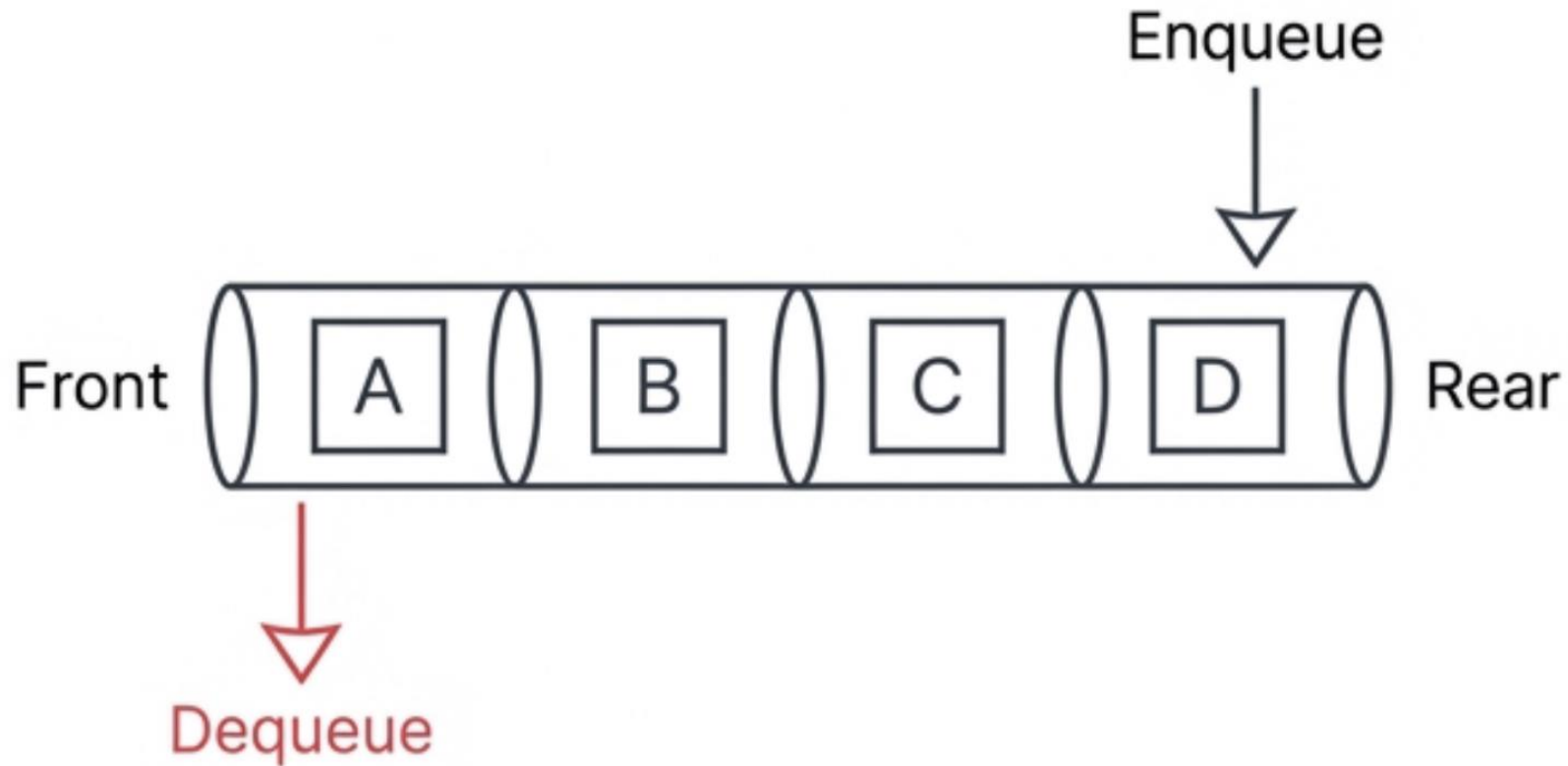


El concepto más crítico a nivel de hardware. Iterar esta pila fuerza a la CPU a buscar nodos no contiguos en el Heap, destruyendo la eficiencia de la Caché L1/L2. En el metal, esto es físicamente más lento que un arreglo.

ArrayStack vs LinkedStack

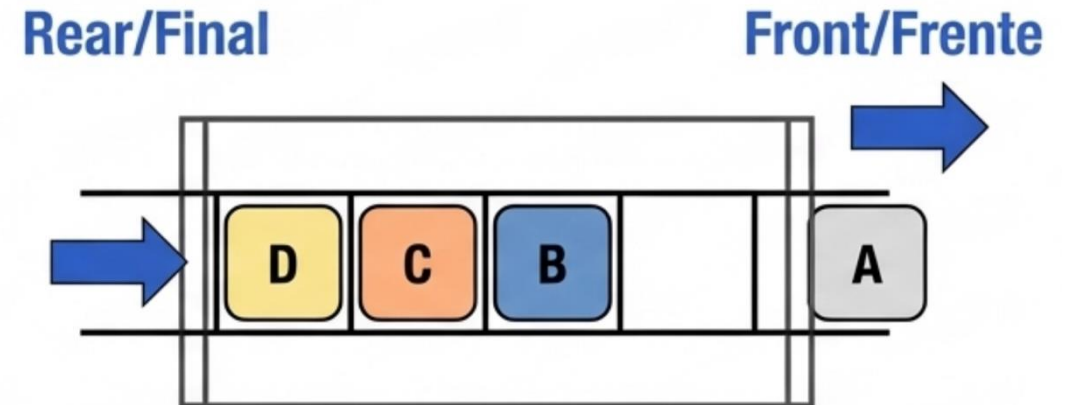
	ArrayStack	LinkedStack
Límite de Capacidad	Rígido (Pre-asignado)	Flexible (Memoria Física)
Complejidad Temporal	$O(1)$ Amortizado (Picos)	$O(1)$ Garantizado
Sobrecarga de Datos	Mínima	Alta (Puntero por nodo)
Eficiencia de Caché (CPU)	Alta (Contigua)	Baja (Fragmentación/Cache Miss)

Cola (Queue) - FIFO



Cola (queue)

A diferencia de la pila (LIFO), la cola opera bajo equidad algorítmica estricta: el primer elemento insertado en la estructura es absolutamente el primero en ser procesado y extraído.



Cola (queue)

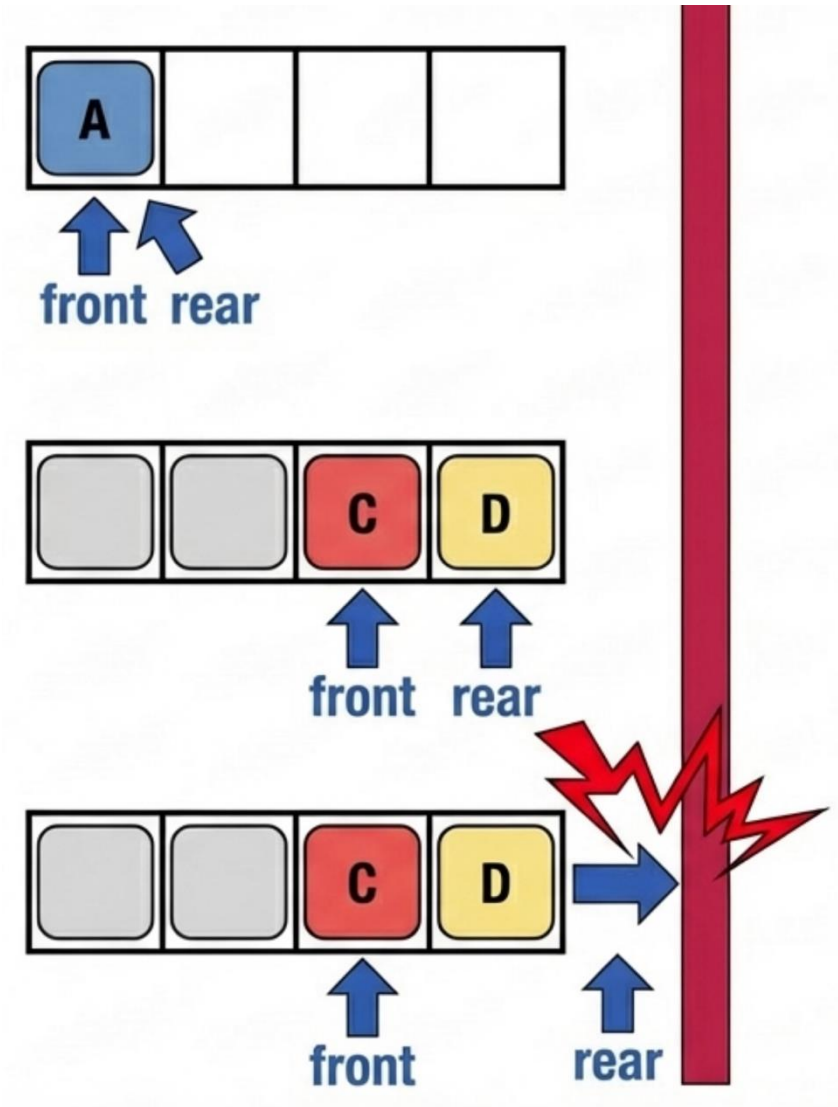
```
template <typename T>
class IQueue {
    virtual void enqueue(const T& element) = 0;
    virtual T dequeue() = 0;
    virtual T front() const = 0;
    virtual bool isEmpty() const = 0;
};
```

Estricto: $O(1)$

Estricto: $O(1)$

Cola (queue)

Falsa Saturación (False Overflow). En una implementación estática simple, los punteros sólo avanzan. Eventualmente, rear alcanza el límite de capacidad y lanza un error de desbordamiento, ignorando las celdas vacías al inicio.



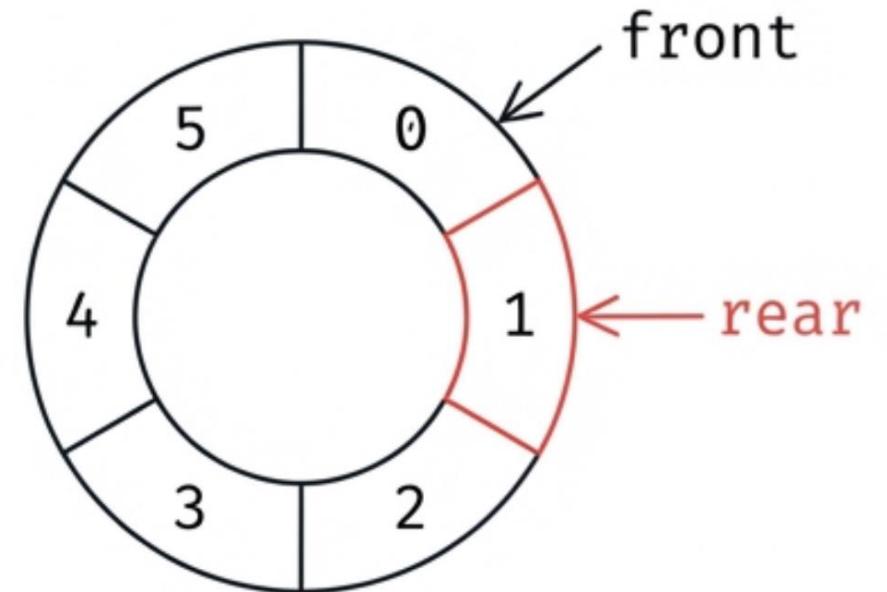
Cola (queue)

Desplazar todos los elementos hacia la izquierda para compactar el arreglo tomaría un tiempo proporcional al número de datos. Esta operación destruye nuestro contrato fundacional de complejidad.



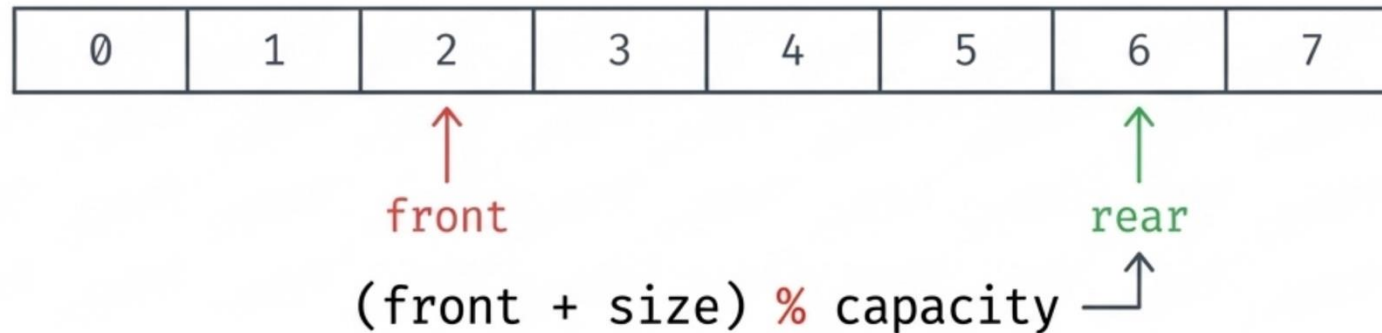
Cola circular (circular queue)

- **Solución Geométrica:** Tratar el bloque de memoria contigua estática como un ciclo lógico cerrado.
- **Reutilización de Búfer:** Cuando el puntero rear alcanza el final del espacio físico, se envuelve automáticamente hacia el índice 0.
- **Resultado:** Eliminación total de la falsa saturación. Reutilización dinámica en $O(1)$ sin desplazamiento masivo de elementos.



Cola circular (circular queue)

El enrutamiento de punteros se logra en tiempo constante mediante el operador módulo (%), evitando condicionales costosos en la CPU.



Cálculo del Rear (Inserción):

```
rear = (front + size) % capacity;
```

(Calcula dinámicamente la posición final basándose en el tamaño actual)

Avance del Front (Extracción):

```
front = (front + 1) % capacity;
```

(Avanza el puntero de salida, reiniciándolo a 0 instantáneamente si equivale a la capacidad máxima)

Cola circular (enqueue)

```
template <typename T>
void QueueArray<T>::enqueue(const T item) {
    // 1. Verificar capacidad para evitar desbordamiento (Overflow)
    if (size + 1 > capacity) {
        resize();
    }

    // 2. Calcular la posición circular del final (rear)
    size_t rear = (front + size) % capacity;

    // 3. Insertar y actualizar estado
    arr[rear] = item;
    ++size;
}
```

Nota Arquitectónica: Calcular rear al vuelo mediante aritmética modular elimina la necesidad de mantenerlo como una variable separada propensa a desincronización.

Cola circular (dequeue)

```
template <typename T>
T QueueArray<T>::dequeue() {
    // 1. Garantizar que la cola no esté vacía (Prevención Underflow)
    assert(size > 0);

    // 2. Extraer el elemento frontal actual
    T item = arr[front];

    // 3. Avanzar el frente circularmente
    front = (front + 1) % capacity;
    --size;

    // 4. (Opcional) Reducir memoria dinámica si sobra mucho espacio
    if (size > 0 && 3 * size < capacity) resize();

    return item;
}
```

Cola circular (observadores)

```
// Retorna referencia al elemento próximo a salir
template <typename T>
T& QueueArray<T>::front_element() const {
    assert(size > 0);
    return arr[front];
}
```

```
// Retorna referencia al último elemento insertado
template <typename T>
T& QueueArray<T>::back_element() const {
    assert(size > 0);
    size_t rear = (front + size - 1) % capacity;
    return arr[rear];
}
```

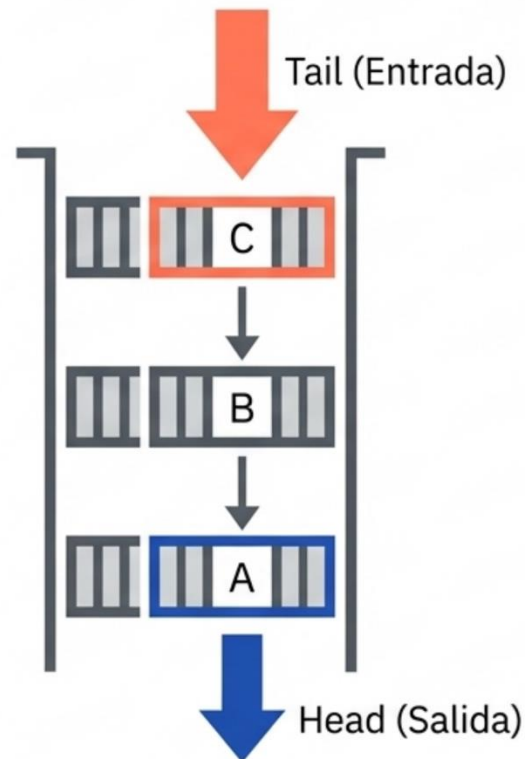
Cola circular (observadores)

```
// Retorna la cantidad total de elementos encolados
template <typename T>
int QueueArray<T>::get_size() const {
    return size;
}
```

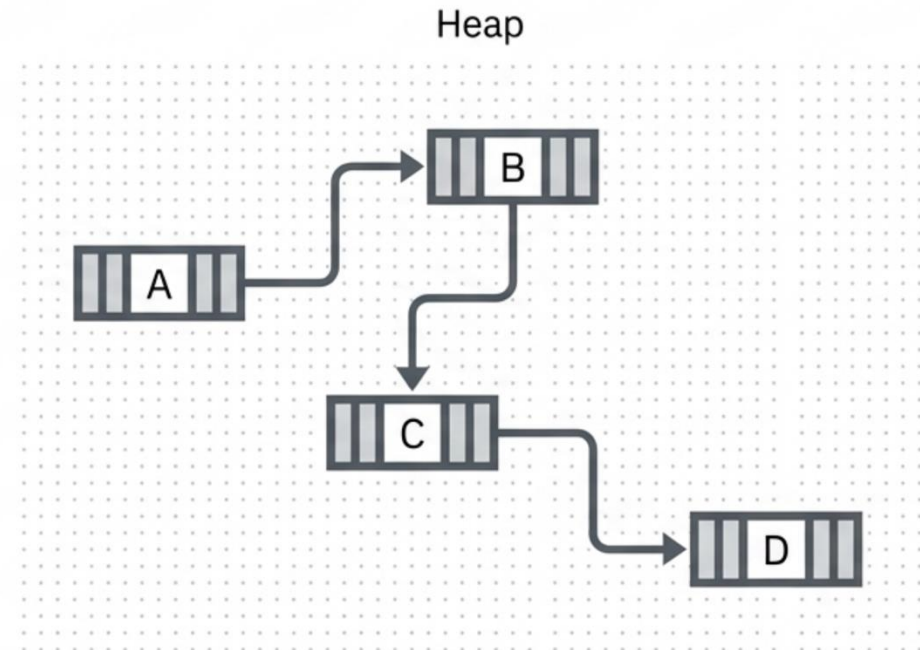
```
// Verifica si la estructura carece de elementos
template <typename T>
bool QueueArray<T>::is_empty() const {
    return (size == 0);
}
```

Cola dinámica (linkedQueue)

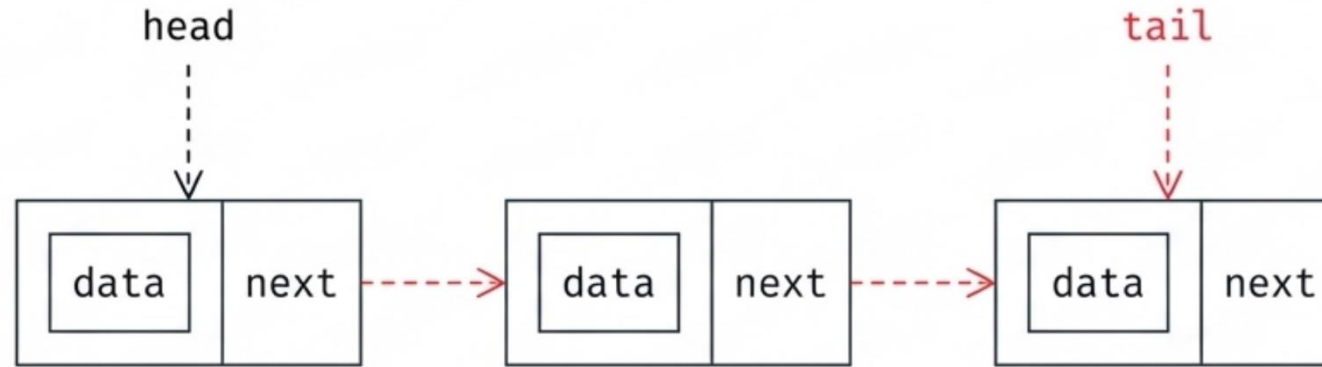
First-In, First-Out. Los datos fluyen como en una tubería.



A diferencia de un arreglo, el Heap proporciona nodos individuales bajo demanda. Sin límites de capacidad. Sin penalización por redimensionamiento.



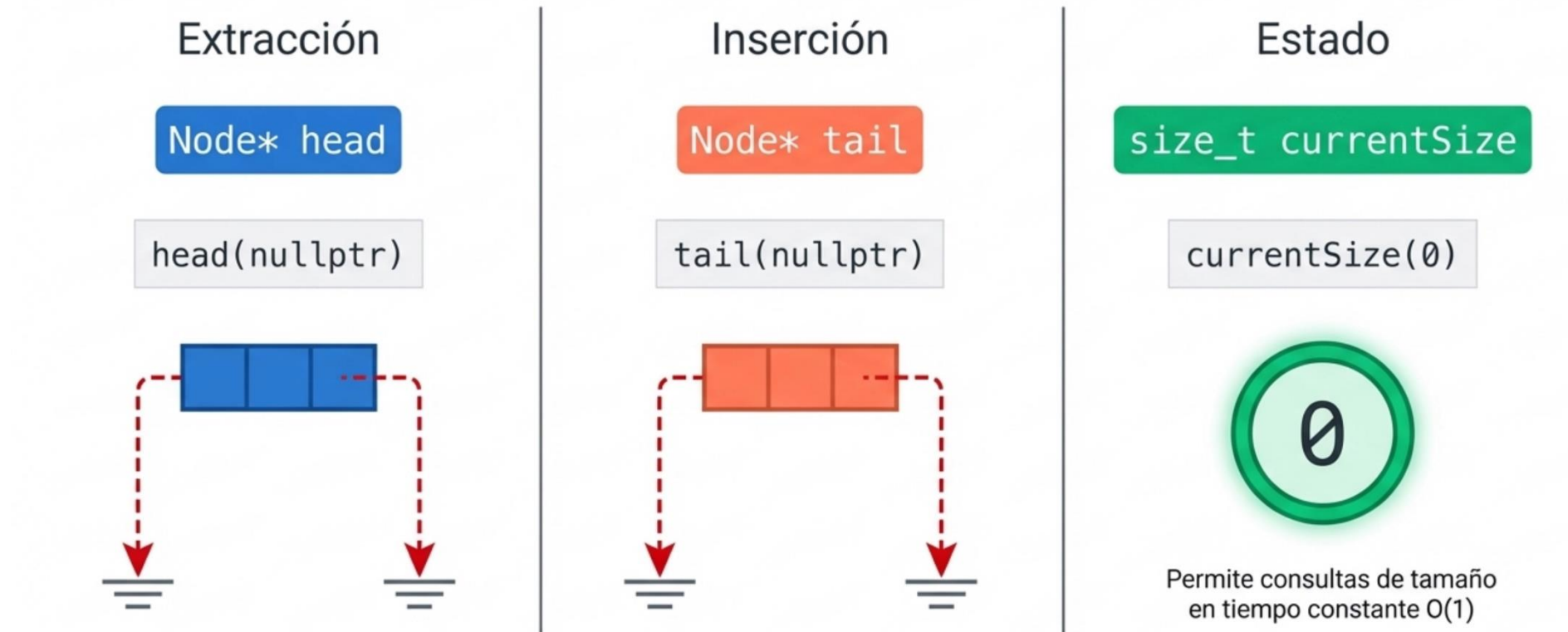
Cola dinámica (LinkedQueue)



- En una lista simple estándar, encolar al final exige recorrer todos los nodos ($O(n)$).
- Introducir un puntero tail que reference constantemente el último nodo transforma la inserción final en una operación instantánea.
- Mapeo de Operaciones:
 - Dequeue (Extracción): Ocurre en el head mediante `popFront()`.
 - Enqueue (Inserción): Ocurre en el tail mediante `pushBack()`

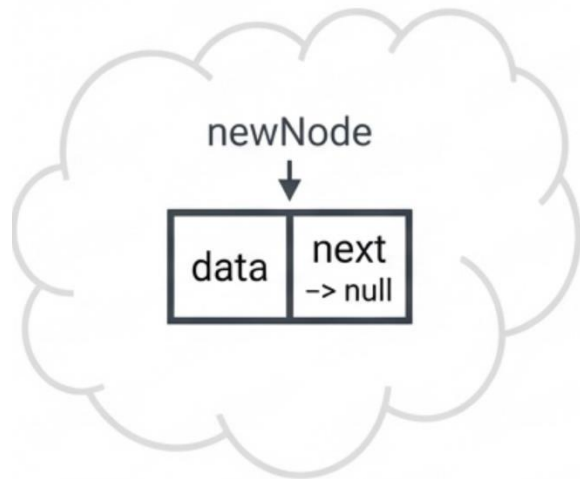
Cola dinámica (LinkedQueue)

Una cola vacía inicializa sus punteros en nulo y su contador en cero.

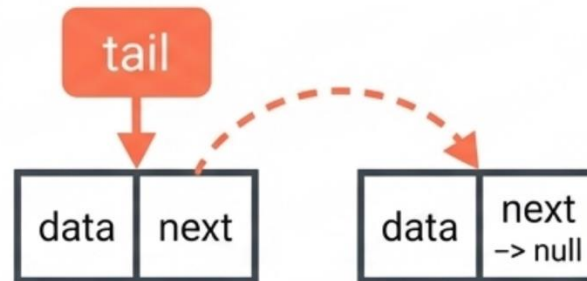


Cola dinámica (enqueue)

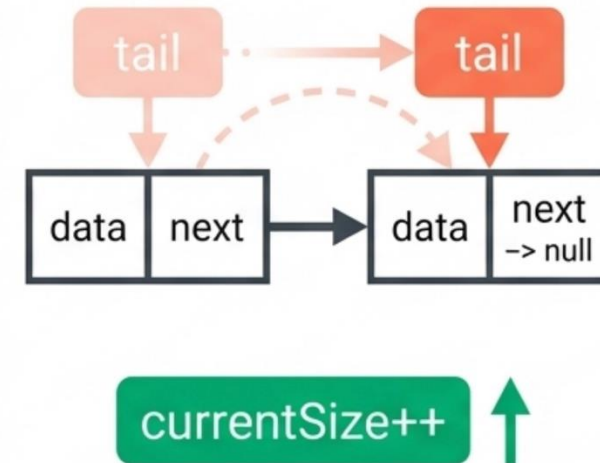
Paso 1: Asignar



Paso 2: Enlazar



Paso 3: Avanzar



Cola dinámica (enqueue)

Bifurcación para
inicializar head y tail
simultáneamente si la
cola está vacía.

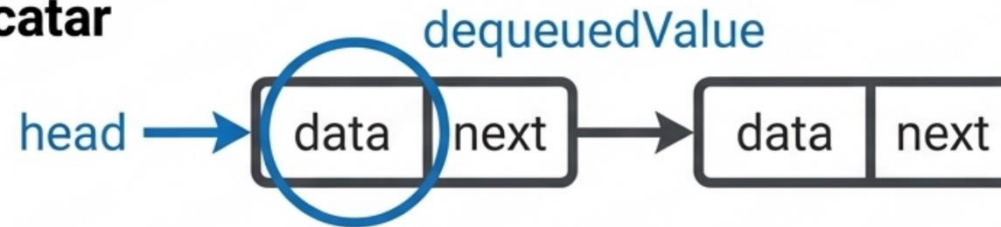
```
void enqueue(const T& element) override {  
    Node* newNode = new Node(element);  
    if (isEmpty()) {  
        head = tail = newNode;  
    } else {  
        tail->next = newNode;  
        tail = newNode;  
    }  
  
    currentSize++;  
}
```



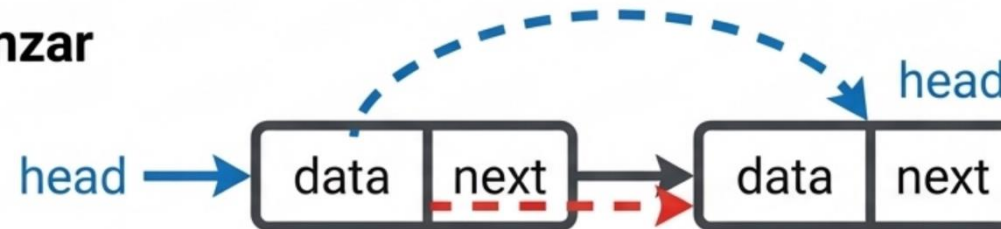
Riesgo documentado:
Excepción `std::bad_alloc`
si el SO deniega la
memoria.

Cola dinámica (dequeue)

Paso 1: Rescatar



Paso 2: Avanzar



Paso 3: Destruir



currentSize-- ↓

Cola dinámica (dequeue)

```
T dequeue() override {
    if (isEmpty()) throw std::underflow_error("Queue
    Underflow.");

    Node* nodeToDelete = head;
    T dequeuedValue = nodeToDelete->data;

    head = head->next;
    if (head == nullptr) { tail = nullptr; }

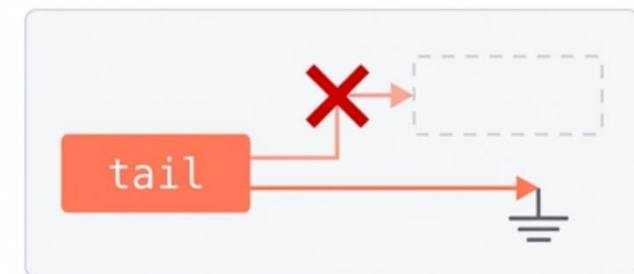
    delete nodeToDelete;
    currentSize--;
    return dequeuedValue;
}
```

Prevencción de Underflow:

Protege contra extracciones en memoria vacía.

Rescate de Dato: Almacena el valor antes de destruir el contenedor físico.

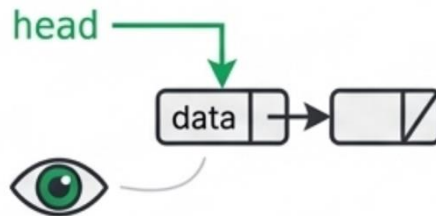
Prevencción de Dangling Pointer



Cola dinámica (observadores)

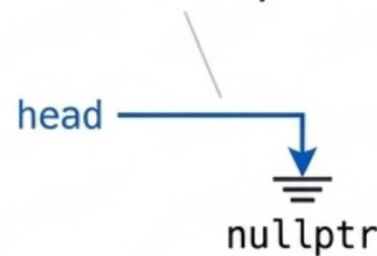
T front() const

Retorna head->data
protegiéndose con
std::underflow_error.



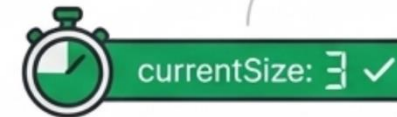
bool isEmpty() const

Evaluación en tiempo
constante $O(1)$: comprueba
si head == nullptr.



size_t getSize() const

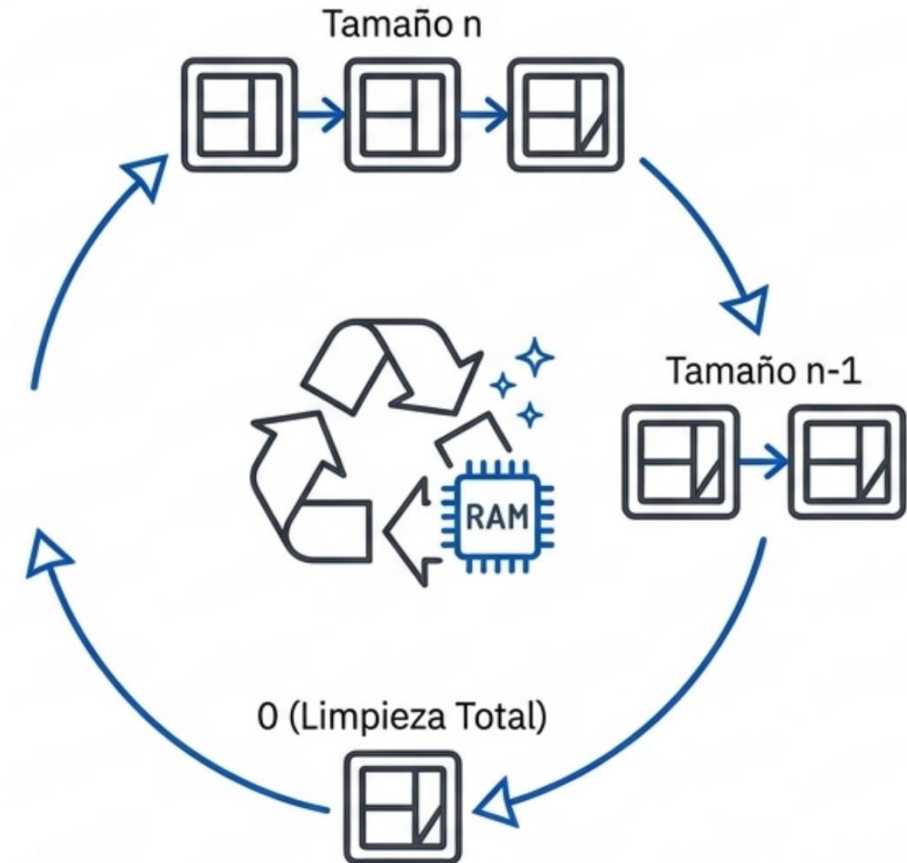
Retorna la variable de
estado **currentSize**,
evitando un recorrido lineal
de conteo $O(N)$.



Cola dinámica (destructor)

Almacenar en el Heap exige limpieza iterativa explícita. El destructor consume la cola hasta vaciarla, previniendo Memory Leaks al destruir la instancia.

```
~LinkedList() override {  
    while (!isEmpty()) {  
        dequeue();  
    }  
}
```



Cola (queue)

LinkedQueue $\mathcal{O}(1)$

Peor Caso

Gracias a la arquitectura de doble puntero (head y tail), la inserción final (pushBack) es una asignación directa de memoria, al igual que la extracción frontal.

CircularQueue $\mathcal{O}(1)$

Caso Promedio / Amortizado

El cálculo de índices mediante la aritmética (front + 1) % capacity es una operación de ciclo único en CPU. La lectura en caché es inmediata y todas las fallas de saturación lineal quedan resueltas axiomáticamente.

CircularQueue vs LinkedQueue

Dimensión	Array Queue	LinkedQueue
Complejidad Enqueue	Amortizada $O(1)$ (Picos en resize)	Estricta $O(1)$ (Predecible) ✓
Uso de Memoria	Contigua (Bloques grandes)	Dispersa en Heap (+Overhead de puntero next)
Localidad de Caché	Alta (Lectura rápida en hardware) ✓	Baja (Saltos de memoria reducen el rendimiento del caché CPU)

Síntesis: Use LinkedQueue para **latencia predecible estricta**. Evítela en procesamiento **masivo intensivo en CPU** debido a la penalización por **localidad de caché**.